



中国矿业大学
CHINA UNIVERSITY OF MINING AND TECHNOLOGY

课程实验报告

基于 SAM 3.1 的开放词汇视频目标分割实验

模型结构、核心代码、运行流程与多提示词实验分析

课程名称:	图像处理与计算机视觉
任课教师:	邵志文
学生姓名:	卜炜珏
学号:	08231338
班级:	计科 3 班
完成日期:	2026 年 6 月

摘要

本次作业选用 Meta 官方开源的 SAM 3.1, 在一段足球短视频上完成开放词汇目标分割与跨帧跟踪, 并补充若干单帧图像提示实验。实验采用官方 SAM3 代码仓库, 模型权重从 ModelScope 的 `facebook/sam3.1` 仓库下载, 运行环境为 NVIDIA GeForce RTX 4090、Python 3.12 和 PyTorch 2.6.0+cu124。报告重点分析项目代码结构、PyTorch 模块构建、checkpoint 加载、推理上下文、Object Multiplex 张量映射、可视化输出流程和本地实验脚本, 并使用 `person`、`ball`、`player in red` 三组视频文本提示词完成对比实验。实验结果显示, SAM 3.1 能根据自然语言提示发现不同粒度的目标实例, 并在连续视频帧中保持较稳定的目标编号和分割结果; 单帧实验进一步展示了场景级、物体级和部件级提示的输出差异。

关键词: SAM 3.1; 开放词汇分割; 视频目标跟踪; Object Multiplex; PyTorch

目录

第一章 实验项目与研究背景	1
1.1 项目选择	1
1.2 实验目标	1
1.3 实验环境	1
第二章 SAM 3.1 项目代码结构解读	2
2.1 官方仓库结构	2
2.2 模型结构	3
第三章 核心代码分析	4
3.1 官方 PyTorch 源码调用链概览	4
3.2 模型构建入口	4
3.3 PyTorch 推理上下文与混合精度	6
3.4 Session 式视频推理	7
3.5 交互式 tracking 源码流程	8
3.6 输出数据结构与 PyTorch 后处理	9
3.7 Object Multiplex 跟踪逻辑	10
3.8 可视化流程	11
3.9 本地实验脚本	12
3.10 单帧图像实验脚本	13
第四章 实验环境与运行流程	13
4.1 环境配置命令	13
4.2 权重下载	14
4.3 实验运行命令	14
第五章 多组实验结果与对比分析	15
5.1 视频实验结果汇总	15
5.2 粗粒度类别提示: person	17
5.3 小目标提示: ball	17
5.4 细粒度属性提示: player in red	17
5.5 单帧图像补充实验	17
5.6 结果对比小结	18
第六章 问题处理与总结	19

6.1	运行中遇到的问题	19
6.2	实验结论	19
6.3	后续改进	19

第一章 实验项目与研究背景

1.1 项目选择

项目选用 Meta 的 SAM 3: Segment Anything with Concepts, 它是 SAM 系列的第三代统一分割基础模型, 也是本次课程作业中的图像处理深度学习开源项目。官方仓库位于 GitHub[Meta AI Research(2026a)], 项目页说明了模型定位并提供了示例演示 [Meta AI Research(2026b)]。本次实验的脚本、报告和结果截图整理在个人公开仓库中 [Bu(2026)], 官方源码和模型权重不随作业仓库上传。SAM 3 支持文本、点、框、mask 等多种提示方式。相比 SAM 2, SAM 3 的关键变化是增加了开放词汇概念分割能力: 用户输入简短文本短语后, 模型可以自动检测并分割相关实例。

SAM 3.1 是 2026 年 3 月发布的更新版本, 重点引入 Object Multiplex, 用共享记忆方式联合处理多个视频目标, 降低多目标跟踪中的重复计算 [Meta AI Research(2026)]。实验中直接加载 `sam3.1_multiplex.pt` checkpoint, 用文本提示词在足球视频帧序列中完成分割与跟踪。

1.2 实验目标

实验工作分为三块。工程部分是在本地 GPU 上配置独立 Conda 环境、下载权重并跑通 SAM 3.1 推理; 代码部分从 PyTorch 源码角度梳理仓库结构、模型构建入口、`state_dict` 加载、`session` 推理接口、Object Multiplex 张量映射、可视化工具和本地脚本改造; 结果部分以视频目标分割为主, 配合单帧图像提示实验, 观察粗粒度类别、小目标、属性约束和部件级提示的输出差异。

1.3 实验环境

依赖安装在独立环境 `sam31` 中, 没有写入 `base` 环境。由于本机 NVIDIA 驱动显示 CUDA 12.4, 实验没有直接使用官方 README 中的 CUDA 12.8 PyTorch 命令, 而是安装与当前驱动兼容的 PyTorch 2.6.0+cu124 预编译包 [PyTorch Contributors(2026)]。环境配置见表 1。

表 1 实验运行环境

项目	配置
操作系统	Linux
Conda 环境	sam31
Python	3.12.13
GPU	NVIDIA GeForce RTX 4090
GPU 显存	49140 MiB
PyTorch	2.6.0+cu124
TorchVision	0.21.0+cu124
CUDA Runtime	12.4
模型权重	sam3.1_multiplex.pt, 来自 ModelScope[ModelScope(2026)]

第二章 SAM 3.1 项目代码结构解读

2.1 官方仓库结构

官方仓库采用标准 Python 包结构。根目录提供说明文档、发布说明、包配置和示例资源，核心实现位于 `sam3/` 包内。主要目录见表 2。

表 2 SAM3 官方仓库结构说明

路径	作用
README.md	项目介绍、环境安装、基础图像和视频推理示例。
RELEASE_SAM3p1.md	SAM 3.1 发布说明，介绍 Object Multiplex、效率提升和新 checkpoint。
pyproject.toml	Python 包配置和基础依赖。
examples/	官方 notebook 示例，包括图像预测、视频预测、批量推理和 SAM 3.1 multiplex 示例。
assets/	官方示例图片、GIF、模型结构图和效率图。
sam3/model_	模型构建入口，封装图像模型、视频模型和 SAM
builder.py	3.1 multiplex predictor。
sam3/model/	核心模型模块，包括编码器、检测器、跟踪器、multiplex 跟踪和 session predictor。
sam3/sam/	SAM 系列基础模块，包括 prompt encoder、mask decoder 和 Transformer。

路径	作用
sam3/ visualization_ utils.py	将分割 mask、边界框和目标编号绘制到原始图像上的工具。
scripts/eval/	benchmark 评估、数据下载和格式转换脚本。

2.2 模型结构

SAM 3 由 detector 和 tracker 组成，二者共享视觉编码器。detector 负责根据文本、几何提示和图像特征发现目标实例，tracker 负责在视频帧之间传播目标记忆并输出连续的 masklet。官方结构如图 1 所示。

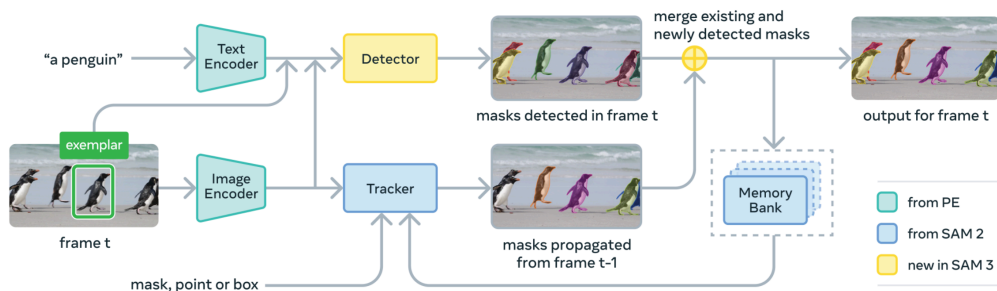


图 1 SAM 3 官方模型结构图。模型将视觉特征、文本提示和几何提示融合，用 detector 输出实例，再由 tracker 完成视频传播。

SAM 3.1 的 Object Multiplex 主要针对视频跟踪阶段。原始 SAM 3 对每个目标相对独立地处理记忆，目标数增加时容易产生重复计算；SAM 3.1 将多个对象组织到固定容量的 bucket 中联合传播，减少记忆读取与更新的重复开销。图 2 展示了官方的 multiplex 思路。

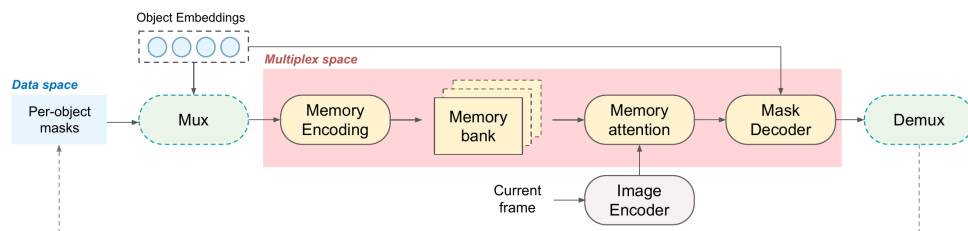


图 2 SAM 3.1 Object Multiplex 示意图。多个对象共享一组记忆处理流程，以提高多目标视频跟踪效率。

第三章 核心代码分析

3.1 官方 PyTorch 源码调用链概览

SAM 3.1 的视频推理由模型构建、请求分发、交互式跟踪和 multiplex 张量转换四个模块依次协作完成，不存在一个函数包揽全部逻辑的入口。表 3 按实验实际执行顺序整理了主要源码位置。报告中的本地脚本只负责抽帧、传参和保存结果，核心 detector、tracker、session、Object Multiplex 和 PyTorch 张量计算逻辑均来自官方仓库。

表 3 SAM 3.1 视频推理源码调用链

源码位置	关键类或函数	在实验中的作用
sam3/model_builder.py	build_sam3_multiplex_video_predictor	组装多个 nn.Module 子模块，加载 checkpoint，并返回统一 predictor。
sam3/model/sam3_base_predictor.py	Sam3BasePredictor	提供 handle_request 和 handle_stream_request，把外部请求转发给底层模型。
sam3/model/sam3_multiplex_video_predictor.py	Sam3MultiplexVideoPredictor	保存 session 参数、默认阈值和异步加载配置，是视频实验直接使用的 predictor 类型。
sam3/model/sam3_multiplex_tracking.py	Sam3MultiplexTrackingWithInteractivity	处理文本、点、框等交互提示，并决定全量传播、局部传播或读取缓存结果。
sam3/model/multiplex_utils.py	MultiplexState、MultiplexController	维护 object 到 bucket/slot 的映射，用 PyTorch 矩阵乘法实现 mux 和 demux。
sam3/model/video_tracking_multiplex.py	VideoTrackingDynamicMultiplex 相关方法	在 mask decoder、memory encoder 和 object pointer 更新中实际执行张量 reshape、拼接、索引和矩阵映射。

从实验角度看，关注点并不在于某层网络的全部参数，而是这些 PyTorch 模块如何把文本提示逐步转化为跨帧 mask 输出。后续分析因此围绕 nn.Module 组装、state_dict 加载、推理上下文、张量 shape 变化和后处理输出展开。

3.2 模型构建入口

模型构建的入口位于 sam3/model_builder.py，核心函数是 build_sam3_multiplex_video_predictor。从 PyTorch 角度看，这个函数的主要工作是把 detector、tracker、text encoder、mask decoder 等多个 nn.Module 子模块拼成一个可推理的组合模型，再把 checkpoint 中的权重张量加载到对应模块。

Listing 1 SAM 3.1 multiplex predictor 构建示例

```

1 from sam3.model_builder import build_sam3_multiplex_video_predictor
2
3 predictor = build_sam3_multiplex_video_predictor(
4     checkpoint_path="checkpoints/sam3.1/sam3.1_multiplex.pt",

```



```

5     use_fa3=False,
6     compile=False,
7     warm_up=False,
8     async_loading_frames=False,
9 )

```

构造函数先创建底层的 multiplex video model，再将 tracker 自身的 backbone 置空并交由交互式模型统一管理，最后通过 Sam3MultiplexPredictorWrapper 标记为 multiplex 动态模式。这样做的目的是避免 tracker 和 demo model 各维护一份独立的 backbone。随后，代码创建 detector 侧的 SAM3VLBackboneTri、transformer、segmentation head、geometry encoder 和 scoring 模块，并把它们作为子模块传入 Sam3MultiplexDetector。

构造函数最后把 detector 与 tracker 包装进交互式 tracking 模块，并设置检测阈值、NMS 阈值、关联 IoU 阈值、新目标阈值、热启动参数、重建参数和 max_num_objects。因此，build_sam3_multiplex_video_predictor 并不只是读一个权重文件，而是把开放词汇检测和视频跟踪两条 PyTorch 计算路径装配成一个可交互系统。

Listing 2 模型构建与 checkpoint 键名映射的源码逻辑摘录

```

1 tracker_model = build_sam3_multiplex_video_model(...)
2 del tracker_model.backbone
3 tracker_model.backbone = None
4
5 tracker = Sam3MultiplexPredictorWrapper(
6     model=tracker_model,
7     is_multiplex=True,
8     is_multiplex_dynamic=True,
9 )
10 detector = Sam3MultiplexDetector(...)
11 demo_model = Sam3MultiplexTrackingWithInteractivity(
12     tracker=tracker,
13     detector=detector,
14     max_num_objects=max_num_objects,
15     recondition_every_nth_frame=16,
16 )
17
18 ckpt = torch.load(checkpoint_path, map_location="cpu", weights_only=True)
19 needs_remap = any(
20     k.startswith("sam3_model.") or k.startswith("sam2_predictor.") for k in ckpt
21 )
22 if needs_remap:
23     remapped_ckpt = {}
24     for k, v in ckpt.items():
25         if k.startswith("sam3_model."):

```

```

26     k = "detector." + k[len("sam3_model."):]
27     elif k.startswith("sam2_predictor."):
28         k = "tracker." + k[len("sam2_predictor."):]
29     remapped_ckpt[k] = v
30     ckpt = remapped_ckpt
31 demo_model.load_state_dict(ckpt, strict=False)
32 predictor = Sam3MultiplexVideoPredictor(model=demo_model, ...)

```

源码先调用 `torch.load(..., map_location="cpu", weights_only=True)`, 将 checkpoint 读到 CPU, 避免一开始就占用 GPU 显存。随后使用 `load_state_dict(strict=False)` 将权重映射到 `demo_model`。这里的 `strict=False` 允许加载过程返回 `missing_keys` 和 `unexpected_keys`, 便于兼容旧命名或不同发布格式, 但不会绕过形状不匹配。最后的 `demo_model.cuda().eval()` 是标准 PyTorch 推理设置: 模型参数移入 GPU, 并把 BatchNorm、Dropout 等模块切到评估模式。

实验关闭了 `use_fa3`, 因为 FlashAttention 3 是可选加速组件, 不参与正确性验证。`max_num_objects` 和 `multiplex_count` 均使用默认值 16, 与官方 checkpoint 参数形状匹配; 若手动改为 8, 加载时 checkpoint 中对应权重的维度与模型定义不一致, 无法通过 `load_state_dict` 检查。

3.3 PyTorch 推理上下文与混合精度

SAM3.1 的推理路径显式使用了 PyTorch 的推理上下文和混合精度机制。请求分发函数 `handle_request` 与 `handle_stream_request` 均带有 `@torch.inference_mode()` 装饰器, 交互式 tracking 的 `init_state`、`add_prompt`、`propagate_in_video` 也使用同类装饰。按照 PyTorch 文档 [PyTorch Contributors(2026)], `inference_mode` 会关闭梯度追踪和 `autograd` 相关状态, 减少推理阶段不必要的内存记录。

Listing 3 推理上下文与混合精度相关源码摘录

```

1 class Sam3MultiplexVideoPredictor(Sam3BasePredictor):
2     def __init__(self, model, ...):
3         self.model = model
4         torch.backends.cuda.matmul.allow_tf32 = True
5         torch.backends.cudnn.allow_tf32 = True
6         self.bf16_context = torch.autocast(
7             device_type="cuda",
8             dtype=torch.bfloat16,
9         )
10        self.bf16_context.__enter__()
11
12    @torch.inference_mode()
13    def handle_request(self, request):

```

```

14     ...
15
16 with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
17     frame_idx, outputs = self.model.add_prompt(**filtered_kwargs)

```

这段源码说明官方实现不是在普通 Python 函数里直接跑网络，而是在 PyTorch 推理环境下执行：TF32 被允许用于 Ampere 架构 GPU 的矩阵乘法 and cuDNN 计算，`torch.autocast(device_type="cuda", dtype=torch.bfloat16)` 则让合适的 CUDA 算子自动使用 bfloat16。对本实验的 RTX 4090 来说，这可以降低显存压力并提高吞吐；同时，由于最后输出会转为 mask、box 和 score，这些推理优化不改变脚本对外拿到的数据结构。

3.4 Session 式视频推理

SAM3 视频 predictor 采用请求分发接口。外部调用不直接访问底层模型，而是向 predictor 发送带有 type 字段的请求。请求分发逻辑位于 `sam3 / model / sam3_base_predictor.py`，常用请求包括 `start_session`、`add_prompt`、`propagate_in_video` 和 `close_session`。

Listing 4 文本提示和跨帧传播的核心调用

```

1 response = predictor.handle_request(
2     request={
3         "type": "add_prompt",
4         "session_id": session_id,
5         "frame_index": 0,
6         "text": "person",
7     }
8 )
9
10 outputs_per_frame = {0: response["outputs"]}
11 for response in predictor.handle_stream_request(
12     request={
13         "type": "propagate_in_video",
14         "session_id": session_id,
15     }
16 ):
17     outputs_per_frame[response["frame_index"]] = response["outputs"]

```

这种设计把交互式分割建模为有状态的会话：session 初始化时一次性加载帧序列，之后可以在任意帧上追加文本、点或框提示，再触发视频范围的传播，无需重复加载图像或重建模型状态。源码中 `Sam3BasePredictor.start_session` 会调用底层模型的 `init_state`，并把返回的 `inference_state` 存入 `_all_inference_states`。后续每次请求都通过 `session_id` 找回这份状

态，因此视频帧缓存、目标 id、用户提示历史和 tracker metadata 可以在多轮交互之间保留下来。

Listing 5 请求分发与参数过滤的源码逻辑摘录

```

1 def handle_request(self, request):
2     if request["type"] == "add_prompt":
3         return self.add_prompt(
4             session_id=request["session_id"],
5             frame_idx=request["frame_index"],
6             text=request.get("text"),
7             points=request.get("points"),
8             bounding_boxes=request.get("bounding_boxes"),
9         )
10
11 def add_prompt(self, session_id, frame_idx, text=None, **kwargs):
12     session = self._get_session(session_id)
13     inference_state = session["state"]
14     call_args = {
15         "inference_state": inference_state,
16         "frame_idx": frame_idx,
17         "text_str": text,
18         **kwargs,
19     }
20     sig = inspect.signature(self.model.add_prompt)
21     valid_params = set(sig.parameters.keys())
22     call_args = {k: v for k, v in call_args.items() if k in valid_params}
23     frame_idx, outputs = self.model.add_prompt(**call_args)
24     return {"frame_index": frame_idx, "outputs": outputs}

```

基类通过 `inspect.signature` 自动过滤不支持的参数，使同一套 `handle_request` 可以兼容 SAM3 与 SAM3.1 之间签名略有差异的底层方法。本地实验脚本中只构造统一的 `request` 字典即可工作，原因正在于此。

3.5 交互式 tracking 源码流程

真正处理 SAM 3.1 视频交互的是 `sam3_multiplex_tracking.py` 中的交互式 tracking 类。它继承自 `Sam3MultiplexTracking`，在基础视频 grounding 上增加了用户行为记录、局部细化和缓存读取逻辑。初始化时，`init_state` 会在基础状态中加入 `action_history`；如果 tracker 处于 per-object 模式，还会初始化内部 SAM2-style 跟踪状态。本实验使用的是 multiplex 动态模式，因此多目标状态主要由 tracker metadata 和 multiplex state 管理。

`add_prompt` 会根据提示类型走两条路径：如果输入点提示，则调用 `add_sam2_new_points` 对已有目标做局部细化；如果输入文本或框提示，则临

时关闭 `use_batched_grounding`，再调用父类的 SAM3 grounding 流程。文本提示关闭 `batched grounding`，是因为这一分支主要依赖 `detector` 的开放词汇检测结果；点提示通常用于已有目标的局部修正，更接近 SAM2 风格的 `per-object` 细化。实验中的 `person`、`ball` 和 `player in red` 都属于文本提示，流程是 `detector` 根据文本生成候选目标，后处理得到初始 `mask` 和 `object id`，再把这些目标写入视频状态。

`propagate_in_video` 在传播前会读取 `action_history`，判断当前应执行全量 `video grounding`、局部 SAM2 传播，还是直接返回已有缓存。首次文本提示后通常进入全量传播路径；如果后续用户只用点提示修正某个对象，则源码可以只传播被修正的目标，并与已有 `video grounding` 结果合并。全量传播内部会继续调用 `video_tracking_multiplex.py` 中的 `mask decoder` 和 `memory encoder`，此时 `multiplex state` 负责完成 `bucket` 与目标索引之间的张量转换。本实验走的是文本提示后的全量路径，即一次 `grounding` 后传播到每一帧。

3.6 输出数据结构与 PyTorch 后处理

每帧返回的输出字典包含四个核心字段，见表 4。`out_obj_ids` 为每个目标分配编号，`out_binary_masks` 和 `out_boxes_xywh` 分别给出分割区域和归一化边界框，`out_probs` 则提供置信度，用于按阈值筛选或排序。

表 4 SAM 3.1 视频输出字段

字段	含义	后续用途
<code>out_binary_masks</code>	每个实例对应的二值分割 <code>mask</code>	叠加到原始帧形成可视化结果。
<code>out_boxes_xywh</code>	归一化边界框，格式为 <code>x、y、w、h</code>	绘制目标框，辅助观察目标位置。
<code>out_probs</code>	每个实例的置信度	在图上显示 <code>score</code> ，帮助判断输出可靠性。
<code>out_obj_ids</code>	跟踪目标编号	观察同一对象在不同帧中的 <code>id</code> 是否稳定。

这些字段来自 `Sam3MultiplexTracking._postprocess_output` 的 PyTorch 后处理。源码先从 `obj_id_to_mask` 字典中取出目标 `id`，并使用 `torch.tensor`、`torch.cat` 将分散的 `mask` 合并为形状为 $N \times H \times W$ 的布尔张量，其中 N 是当前帧保留的目标数。随后用 `out_binary_masks.any(dim=(1, 2))` 删除面积为 0 的 `mask`；如存在被抑制或删除的目标，再用 `torch.isin`、`torch.nonzero` 和 `torch.index_select` 做索引筛选。

Listing 6 输出后处理中的 PyTorch 张量操作摘录

```

1 out_obj_ids = torch.tensor(curr_obj_ids, dtype=torch.int64)
2 out_binary_masks = torch.cat(
3     [obj_id_to_mask[obj_id] for obj_id in curr_obj_ids],
4     dim=0,
5 )
6 keep = out_binary_masks.any(dim=(1, 2)).cpu()
7 keep_idx = torch.nonzero(keep, as_tuple=True)[0]
8 keep_idx_gpu = keep_idx.pin_memory().to(
9     device=out_binary_masks.device,
10    non_blocking=True,
11 )
12 out_binary_masks = torch.index_select(out_binary_masks, 0, keep_idx_gpu)
13 out_boxes_xywh = box_xyxy_to_xywh(masks_to_boxes(out_binary_masks))
14 outputs = {
15     "out_obj_ids": out_obj_ids.cpu().numpy(),
16     "out_boxes_xywh": out_boxes_xywh.cpu().numpy(),
17     "out_binary_masks": out_binary_masks.cpu().numpy(),
18 }

```

这里能看到从 GPU 张量到报告可视化数据的完整转换：mask 的筛选和 box 计算在 PyTorch 张量上完成，最终通过 `.cpu().numpy()` 转成 NumPy 数组，交给可视化函数绘图。边界框先以像素坐标计算，再除以视频宽高归一化，因此表中 `out_boxes_xywh` 的数值可直接用于不同分辨率图像上的显示。

3.7 Object Multiplex 跟踪逻辑

Multiplex 状态管理位于 `multiplex_utils.py`。其中 `MultiplexController` 继承自 `nn.Module`，负责根据目标数和 `multiplex_count` 计算 bucket 数；`MultiplexState` 保存二维 assignments，外层是 bucket，内层是 bucket 中的 slot。普通目标使用非负索引，空 slot 用 -1 表示，被删除目标用 -1116 标记。这样，模型可以把任意数量的目标补齐到 `num_buckets × multiplex_count` 的规则形状。

Listing 7 MultiplexState 中 mux 和 demux 的简化逻辑

```

1 class MultiplexState:
2     def _precompute_transition_matrices(self):
3         mux_matrix = zeros(num_buckets * multiplex_count, total_valid_entries)
4         demux_matrix = zeros(total_valid_entries, num_buckets * multiplex_count)
5         for bucket_id, bucket in enumerate(assignments):
6             for slot_id, object_idx in enumerate(bucket):
7                 if object_idx >= 0:
8                     flat_slot = bucket_id * multiplex_count + slot_id
9                     mux_matrix[flat_slot, object_idx] = 1.0

```

```

10         demux_matrix[object_idx, flat_slot] = 1.0
11
12     def mux(self, x):
13         # data space: [num_objects, ...]
14         # multiplex space: [num_buckets, multiplex_count, ...]
15         return (self.mux_matrix @ flatten(x)).view(mux_shape)
16
17     def demux(self, x):
18         # multiplex space -> data space, padding slots are ignored
19         return (self.demux_matrix @ flatten(x)).view(data_shape)
    
```

源码的核心是两张预计算的 PyTorch 矩阵: `mux_matrix` 的形状为 $(B \times M) \times N$, `demux_matrix` 的形状为 $N \times (B \times M)$, 其中 B 是 bucket 数, M 是 `multiplex_count`, N 是有效目标数。`mux` 先把输入张量 reshape 成 $N \times F$, 再执行 `self.mux_matrix @ x_flat`, 最后 view 成 $B \times M \times \dots$; `demux` 则执行相反方向的矩阵乘法。由于 padding slot 在矩阵中没有有效映射, 空位不会进入最终实例列表。

实际张量转换分布在 `video_tracking_multiplex.py`。在 mask decoder 的 propagation 分支中, `sam_mask_decoder` 输出的 `low_res_multimasks`、`object_score_logits` 和 `sam_output_tokens` 先处在 $B \times M$ 的 multiplex 空间, 随后统一调用 `multiplex_state.demux(...)` 还原为 N 个目标。记忆写入路径则相反: `_encode_new_memory` 中的 `mask_for_mem` 先通过 `multiplex_state.mux(...)` 转成 bucket 形式, 再送入 `maskmem_backbone`。

新增目标时, `add_new_masks_to_existing_state` 还要更新 bucket/slot 分配。源码首先将已有目标的 `obj_ptr` 从 multiplex 空间 demux 到数据空间, 附加新目标的 pointer 张量, 再通过 `torch.cat` 合并为新的 pointer 序列, 最后用 `multiplex_state.mux(combined_pointers)` 写回状态。旧目标本身没有变化, 但它们在 bucket 中的位置可能因动态重分配而变化, 这也是 multiplex controller 需要维护可逆映射的原因。

综上, Object Multiplex 的核心机制是在目标索引、bucket slot、object pointer 和 mask memory 之间维持一组可逆的线性映射矩阵, 而不是简单地将多个 mask 拼成 batch。对外接口仍然返回按目标排列的 `out_obj_ids` 等字段, 对内则通过 bucket 结构复用 memory encoder/decoder 的中间结果, 避免随目标数线性增长的重复操作。

3.8 可视化流程

可视化函数位于 `sam3 / visualization_utils.py`。本实验使用 `save_masklet_image` 将每帧输出保存为图片。该函数内部先读取原始帧,

再调用渲染函数叠加 mask、边界框、object id 和置信度。

Listing 8 单帧可视化保存示例

```
1 save_masklet_image(
2     frame=str(frames[frame_idx]),
3     outputs=outputs_per_frame[frame_idx],
4     out_path=str(output_path),
5     alpha=0.5,
6     frame_idx=frame_idx,
7 )
```

3.9 本地实验脚本

本地脚本 `scripts/run_sam31_player.py` 把官方 notebook 中的推理逻辑抽出来，改成命令行可复现的实验。该脚本不修改任何官方源码，独立完成 GIF 帧抽取、模型加载、session 创建、提示传播和结果保存五个环节，流程图见图 3。

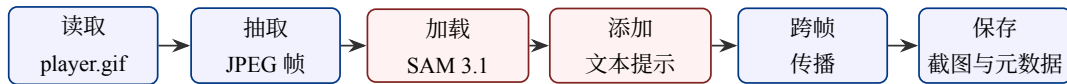


图 3 本地实验脚本推理流程。脚本将视频帧抽取、模型加载、文本提示、跨帧传播和结果保存组织为可重复命令。

脚本还增加了 `--output-prefix` 参数，避免不同提示词实验的图片、metadata 和日志相互覆盖。例如 `person` 对应 `sam31_player_person_summary.png`，`ball` 对应 `sam31_player_ball_summary.png`。

Listing 9 多提示词输出命名逻辑

```
1 parser.add_argument("--prompt", default="person", help="Text prompt.")
2 parser.add_argument(
3     "--output-prefix",
4     default=None,
5     help="Output filename prefix. Defaults to a prompt-based slug.",
6 )
7
8 def slugify_prompt(prompt: str) -> str:
9     slug = re.sub(r"[^a-z0-9]+", "_", prompt.lower()).strip("_")
10    return slug or "prompt"
11
12 output_prefix = args.output_prefix or f"sam31_player_{slugify_prompt(args.prompt)}"
13 summary_path = args.results_dir / f"{output_prefix}_summary.png"
14 metadata_path = args.results_dir / f"{output_prefix}_metadata.json"
```

通过 `--output-prefix` 和 `slugify_prompt`，不同 prompt 的输出文件会自动携带对应的前缀标识，避免相互覆盖。

脚本中还加入了 `start_multiplex_session` 辅助函数，直接调用 `predictor.model.init_state` 并手动登记 `session`。这是为了适配当前 SAM3.1 `multiplex` 模型的参数签名：官方基类 `start_session` 会传入 `offload_state_to_cpu`，而本地安装版本的 `multiplex init_state` 并不接收这个参数。该兼容逻辑只放在实验脚本中，不改官方源码，便于后续更新 SAM3 仓库时直接替换上游代码。

3.10 单帧图像实验脚本

除了视频 `predictor`，本实验还补充了 `scripts/run_sam31_image_cases.py`。该脚本使用 `sam3/model/sam3_image_processor.py` 中的 `Sam3Processor`，在若干静态图片上执行文本提示分割。其核心调用更接近普通图像分割任务：先构建图像模型，再设置输入图像，最后输入文本 `prompt` 得到 `masks`、`boxes` 和 `scores`。

Listing 10 单帧图像文本提示调用

```
1 model = build_sam3_image_model(
2     checkpoint_path=str(args.checkpoint),
3     load_from_HF=False,
4     compile=False,
5 )
6 processor = Sam3Processor(
7     model,
8     confidence_threshold=args.confidence_threshold,
9 )
10
11 state = processor.set_image(image)
12 output = processor.set_text_prompt(state=state, prompt=case.prompt)
```

该脚本用于补充 SAM 3.1 在静态图像上的开放词汇分割表现。实验选取了 `building`、`truck`、`wheel` 等成功样例，以及 `tree` 和 `bottle` 在默认阈值下未保留实例的对照样例，以展示不同提示词粒度和阈值设置对输出结果的影响。

第四章 实验环境与运行流程

4.1 环境配置命令

以下是环境搭建用到的关键命令。安装前确认当前 `shell` 位于 `base` 环境，但所有依赖均通过 `conda run -n sam31` 安装到独立环境。

Listing 11 环境配置命令摘录

```
1 conda create -n sam31 python=3.12 -y
```

```

2
3 conda run -n sam31 python -m pip install \
4     torch==2.6.0 torchvision==0.21.0 \
5     --index-url https://download.pytorch.org/whl/cu124
6
7 conda run -n sam31 python -m pip install -e . \
8     modelscope matplotlib opencv-python-headless imageio
9
10 conda run -n sam31 python -m pip install \
11     einops pycocotools psutil pandas scikit-image scikit-learn

```

由于 SAM3 要求 `numpy<2`，且源码仍使用 `pkg_resources`，实验中固定了两个兼容依赖：

Listing 12 兼容依赖固定

```

1 conda run -n sam31 python -m pip install \
2     --force-reinstall --no-deps numpy==1.26.4
3 conda run -n sam31 python -m pip install "setuptools<81"

```

4.2 权重下载

SAM 3.1 checkpoint 从 ModelScope 下载，避免 Hugging Face gated checkpoint 登录问题。下载命令如下：

Listing 13 ModelScope 权重下载命令

```

1 mkdir -p checkpoints/sam3.1
2 conda run -n sam31 modelscope download \
3     --model facebook/sam3.1 \
4     sam3.1_multiplex.pt config.json \
5     --local_dir checkpoints/sam3.1

```

下载完成后，`sam3.1_multiplex.pt` 文件大小为 3,502,755,717 字节，SHA256 为 0567debeec80ba4ac6369540c6c248025283cb3ff2b92827509e57e2b3541cb6，与 ModelScope LFS 指针一致。

4.3 实验运行命令

三组实验均使用官方 `assets/player.gif` 抽取出的 8 帧序列，帧步长为 6。

Listing 14 三组视频 prompt 实验命令

```

1 PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
2 conda run -n sam31 python scripts/run_sam31_player.py \
3     --max-frames 8 --frame-step 6 \
4     --prompt person --output-prefix sam31_player_person

```

```

5
6 PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
7 conda run -n sam31 python scripts/run_sam31_player.py \
8     --max-frames 8 --frame-step 6 \
9     --prompt ball --output-prefix sam31_player_ball
10
11 PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
12 conda run -n sam31 python scripts/run_sam31_player.py \
13     --max-frames 8 --frame-step 6 \
14     --prompt "player in red" --output-prefix sam31_player_red

```

单帧图像补充实验使用同一个 checkpoint，但走图像 predictor，不创建视频 session：

Listing 15 单帧图像补充实验命令

```

1 conda run -n sam31 python scripts/run_sam31_image_cases.py \
2     --confidence-threshold 0.35 --max-instances 8

```

脚本生成的文件按结果类型分开保存。表 5 列出报告中使用的主要输出，后续复现实验时可以直接对照这些文件检查。

表 5 实验输出文件说明

文件	来源脚本	内容
sam31_player_*_metadata.json	run_sam31_player.py	三组视频实验的帧数、耗时、GPU 和逐帧目标数。
sam31-video-prompt-grid.png	报告整理图	三组视频 prompt 在第 0、4、7 帧的九宫格对比。
sam31_image_cases_metadata.json	run_sam31_image_cases.py	单帧图像实验中各 prompt 的目标数、耗时和最高置信度。
sam31-image-cases-grid.png	报告整理图	单帧图像成功样例和失败样例说明。

第五章 多组实验结果与对比分析

5.1 视频实验结果汇总

三组视频结果汇总在表 6 中。每组都顺利完成 8 帧传播，没有出现空白输出。图 4 将三组 prompt 在第 0、4、7 帧上的输出统一排版，便于直接比较类别提示、小目标提示和属性提示之间的差异。

表 6 三组文本提示词实验结果

Prompt	每帧目标数	加载时间	推理时间	观察结果
person	3	9.35 s	1.46 s	检测并跟踪三名球员，目标 id 在不同帧中保持稳定。
ball	1	9.47 s	1.43 s	检测并跟踪足球，小目标位置能够随帧变化更新。
player in red	1	9.42 s	1.49 s	只选择红色球员，体现文本属性约束能力。



图 4 三组视频文本提示词的跨帧分割结果对比。每一行对应一个 prompt，每一列对应同一视频中的一个采样帧。

Listing 16 player in red 实验 metadata 摘录

```

1 {
2   "prompt": "player in red",
3   "num_frames": 8,
4   "selected_frames": [0, 4, 7],
5   "load_seconds": 9.42,
6   "inference_seconds": 1.49,
7   "objects_per_frame": {
8     "0": 1,
9     "1": 1,
10    "2": 1,
11    "3": 1,
12    "4": 1,
13    "5": 1,
14    "6": 1,

```

```
15     "7": 1
16   }
17 }
```

metadata 中记录了 prompt、采样帧、模型加载耗时、推理耗时和每帧目标数。以上耗时数据均由脚本自动记录至 metadata JSON 文件，随后整理为表格。

5.2 粗粒度类别提示：person

使用 person 提示时只给出人物类别，没有限制颜色或位置。模型将画面中符合人物概念的多个实例都找了出来，因此每帧得到 3 个目标。图 4 第一行可以看到，白衣球员、红衣球员和远处球员被分配不同颜色和 id，目标集合在后续帧中保持稳定。

5.3 小目标提示：ball

足球面积小，和草地背景颜色也比较接近，属于这段视频中较难稳定跟踪的目标。使用 ball 提示时，模型仍能在每帧中保持 1 个足球目标输出。图 4 第二行显示足球框会随运动位置更新，但 mask 面积较小，说明小目标结果对分辨率和置信度阈值更敏感。

5.4 细粒度属性提示：player in red

player in red 在类别基础上加入了颜色属性约束。与 person 不同，该提示并不要求输出全部人物，而是只锁定红色球衣球员。图 4 第三行显示，模型每帧输出 1 个目标，并优先锁定红衣球员。这说明 SAM 3.1 的文本提示不仅能表达类别，也能表达颜色属性等细粒度概念。

5.5 单帧图像补充实验

为进一步验证模型在不同场景下的泛化能力，本实验补充选取校园夜景、车厢内日常物品场景和街景货车图片进行单帧开放词汇分割。图像实验共加载一次模型，加载时间为 6.12 s；每个 case 独立调用 set_image 和 set_text_prompt。结果见表 7 和图 5。

表 7 单帧图像文本提示实验结果

图像场景	Prompt	目标数	推理时间	最高分	观察结果
校园夜景	building	11	0.28 s	0.859	能找出主楼和远处建筑，但夜景噪声使部分检测框较为零散。
校园夜景	tree	0	0.09 s	—	树木与暗背景区分度低，默认阈值下未保留实例。
车厢物品	bottle	0	0.09 s	—	目标被纸袋遮挡且外观不明显，未形成稳定候选。
街景货车	truck	1	0.09 s	0.875	能完整覆盖整车主体。
街景货车	wheel	4	0.09 s	0.949	能响应部件级提示，但也会包含车轮附近相似部件。

SAM 3.1 单帧图像提示补充实验

从场景级、物体级和部件级三个角度观察开放词汇分割输出



building

校园夜景，多实例建筑区域



truck

整车级目标，单实例完整轮廓



wheel

部件级目标，多个车轮相关实例

阈值与提示词敏感性

tree：夜景中树木与暗背景混在一起，在 0.35 阈值下没有保留实例。

bottle：车厢杂物被纸袋遮挡，默认阈值下未形成稳定候选。

这两组写入结果表，作为开放词汇分割受场景、阈值影响的对照样例。

图 5 单帧图像文本提示补充实验。成功样例覆盖场景级建筑、整车目标和车轮部件；右下角记录默认阈值下未保留实例的对照样例。

5.6 结果对比小结

视频实验和单帧实验共同说明了 SAM 3.1 的开放词汇能力。类别提示更容易召回多个实例，例如 person 和 building；细粒度提示能将结果聚焦到特定目标，例如 player in red 和 wheel；小目标或遮挡目标更依赖图像清晰度和阈值设置，例如 ball、tree 和 bottle。因此，SAM 3.1 适合快速完成自然语言驱动的分割实验。如需进一步定量评测，可引入人工标注及 IoU、AP 等指标。

第六章 问题处理与总结

6.1 运行中遇到的问题

实验过程中主要处理了环境版本、依赖缺失、接口兼容和阈值选择四类问题。官方 README 推荐 CUDA 12.8 PyTorch，但本机驱动显示 CUDA 12.4，因此实际采用 torch==2.6.0+cu124。SAM3 源码导入 pkg_resources，新版 setuptools 会导致模块缺失，所以固定 setuptools<81。此外，官方基础依赖没有完全覆盖 video predictor 的实际运行路径，需要额外安装 einops、pycocotools、psutil 等包。SAM3.1 multiplex 模型的 init_state 参数签名与基类 start_session 传入的参数不匹配，本实验脚本通过兼容启动函数规避了该问题，未改动官方源码。单帧图像实验中，tree 和 bottle 在默认阈值下没有输出，也说明开放词汇分割结果仍会受图像质量、遮挡和 confidence threshold 影响。

6.2 实验结论

本实验在本地 RTX 4090 上跑通了 SAM 3.1 的开放词汇视频分割、多目标跟踪和单帧图像提示分割。三组视频提示词分别验证了不同能力：person 对应多实例类别分割，ball 对应小目标定位，player in red 对应属性约束下的细粒度检索；单帧图像实验进一步展示了 building、truck、wheel 等不同粒度 prompt 的输出差异。从 PyTorch 源码角度来看，SAM 3.1 通过 nn.Module 组合、state_dict 权重加载、inference_mode 推理上下文、bf16/TF32 混合精度和 multiplex 矩阵映射，把 detector、tracker 和 memory encoder 等复杂模块封装在 session 请求接口后面；对外调用简洁，但内部数据流仍然是清晰的张量计算过程。

6.3 后续改进

后续可进一步增加自采视频或公开数据集样例，并比较 SAM 3、SAM 3.1 和 SAM 2.1 在相同视频上的速度与精度差异。单帧实验可以增加不同 confidence threshold 的扫描，观察召回率和误检之间的变化。如果环境支持 CUDA 12.8 和 FlashAttention 3，也可以重新开启 use_fa3 与 torch.compile，评估 Object Multiplex 的实际加速效果。

参考文献

- [Meta AI Research(2026a)] Meta AI Research, . SAM 3: Segment Anything with Concepts. <https://github.com/facebookresearch/sam3>, 2026a. Accessed: 2026-06-07.
- [Meta AI Research(2026b)] Meta AI Research, . Segment Anything Model 3 Project Page. <https://ai.meta.com/sam3>, 2026b. Accessed: 2026-06-07.
- [Bu(2026)] Bu, W. SAM 3.1 Vision Homework Experiment Materials. <https://github.com/muqy1818/sam31-vision-homework>, 2026. Accessed: 2026-06-08.
- [Meta AI Research(2026)] Meta AI Research, . SAM 3.1 Release Notes. https://github.com/facebookresearch/sam3/blob/main/RELEASE_SAM3p1.md, 2026. Accessed: 2026-06-07.
- [PyTorch Contributors(2026)] PyTorch Contributors, . PyTorch Installation Documentation. <https://pytorch.org/get-started/locally/>, 2026. Accessed: 2026-06-07.
- [ModelScope(2026)] ModelScope, . facebook/sam3.1 Model Repository. <https://www.modelscope.cn/models/facebook/sam3.1>, 2026. Accessed: 2026-06-07.
- [PyTorch Contributors(2026)] PyTorch Contributors, . PyTorch API Documentation. <https://docs.pytorch.org/docs/2.11/>, 2026. Accessed: 2026-06-08.